層の行列計算をまとめる（-w 5）

- 実験日: 2026 年 10 月 19 日
- 担当: 櫻田
- ブランチ: `dev/sakurada`（実装: コミット `edf51f6`、結果: `212ea59`）
- 機材: 学科 PC EE00025（Intel Core Ultra 5 225U）、ネイティブ Linux（Ubuntu 24.04.3）
- 生データ: `memlab/native_w0345_EE00025_20261019/`（2 回分）

ひとことで言うと

- 層の中で **同じ入力に掛けている行列（wq・wk・wv と、w1・w3）を縦につなげ、1 回の掛け算で計算する** ようにした。
- 1 トークンあたりの行列の掛け算が **42 回 → 24 回** に減り、スレッド同士の待ち合わせの回数も同じだけ減った。
- 計算結果は 1 ビットも変わらない（生成される文章は完全に一致）。読む重みの量も変わらない。
- マルチスレッドでは **層の合計が 8〜12% 速くなった**（1 スレッドでは 2%）。一番速い構成（P4_HT）は、ヒュージページと合わせて **631 → 702 tok/s（約 11% アップ）**。

---

1. 層の中では何を計算しているか

`stories15M` は 6 つの層を持ち、1 つの層では次の順に計算する（x はトークンの今の状態を表す 288 個の数字）。

```
① q = Wq・x     ┐
② k = Wk・x      │ 入力が同じ x（正規化したもの）
③ v = Wv・x     ┘
④ アテンション（q, k, v と過去のトークンから結果を作る）
⑤ Wo・(④の結果)          ← 入力は ④ の結果
⑥ h1 = W1・x'   ┐
⑦ h3 = W3・x'   ┘ 入力が同じ x'（⑤のあとに正規化したもの）
⑧ SwiGLU(h1, h3)
⑨ W2・(⑧の結果)          ← 入力は ⑧ の結果
```

記号	行列の大きさ	役割
Wq, Wk, Wv	288 × 288	アテンションで使う q（探しているもの）、k（見出し）、v（中身）を作る
Wo	288 × 288	アテンションの結果を元の形に戻す
W1, W3	768 × 288	FFN で値を 768 個に広げる
W2	288 × 768	FFN で 288 個に戻す

①②③ は同じ x に掛けていて、⑥⑦ も同じ x' に掛けている。元の `run.c` では、これらを 1 つずつ別々に計算していた（1 層あたり 7 回、1 トークンで 6 層 × 7 回 = 42 回）。

---

2. 何が問題だったか：スレッド同士の待ち合わせ

OpenMP でマルチスレッドにすると、行列の掛け算 1 回ごとに、次の流れが入る。

```
「みんな、この行列を分担して」 → 4 人で計算 → 「全員終わった？」（待ち合わせ）
```

- 層の行列は小さい。288 行を 4 スレッドで分けると、1 人 72 行で、数マイクロ秒で終わる。
- そのため、**始める・待ち合わせる手間が、計算そのものと同じくらいかかっていた**。
- 実際、元の `run.c` では、層の合計は 1 スレッド → 4 スレッド（P1 → P4_HT）で 1.29 倍しか速くならなかった（出力層は 1.56 倍。-w 0、16:05 の測定）。

---

3. -w 5 でやったこと：同じ入力に掛ける行列をつなげる

wq・wk・wv をつなげる

```
   ┌ Wq（288 行）┐            ┌ q（288 個）┐
   │ Wk（288 行）│  ・ x  =   │ k（288 個）│
   └ Wv（288 行）┘            └ v（288 個）┘
 864 行 × 288 列            864 個の答え
```

- 行列の掛け算は **行ごとに独立** している（1 行目の答えは、1 行目と x だけで決まる）。
- つなげても各行の計算はまったく同じで、答えの最初の 288 個が q、次の 288 個が k、最後の 288 個が v になる。

w1・w3 をつなげる

```
   ┌ W1（768 行）┐  ・ x’  =   ┌ h1（768 個）┐
   └ W3（768 行）┘            └ h3（768 個）┘
 1,536 行 × 288 列
```

まとめられないもの

⑤ Wo と ⑨ W2 はまとめられない。入力が直前の計算（④・⑧）の結果なので、その計算が終わるまで始められないため。

回数の変化

	元（-w 4 まで）	-w 5
1 層の行列の掛け算	7 回（①②③⑤⑥⑦⑨）	**4 回**（①〜③ まとめて、⑤、⑥⑦ まとめて、⑨）
1 トークン（6 層）の掛け算	42 回	**24 回**
読む重みの量	23.9MB	**23.9MB（同じ）**

---

4. 実装（run.c）

重みの並べ替え（起動時）

- 行列をつなげるには、Wq・Wk・Wv の各行がメモリ上で続いている必要がある。元のファイルは「6 層分の Wq → 6 層分の Wk → …」と **種類ごと** に並んでいるので、そのままではつながらない。
- そこで、起動時に重みをメモリへコピーするとき、**層ごとに Wq, Wk, Wv, Wo, W1, W3, W2 の順** に並べ替える（`relayout_layers()`）。Wq の直後に Wk・Wv、W1 の直後に W3 が来るので、それぞれ 1 つの大きな行列として扱える。
- **重みファイル（stories15M.bin）は変更しない**。並べ替えた重みも、ヒュージページ（2MB ページ）に置く。
- この並べ替えだけを行い、計算はまとめないモードが `-w 4`。

計算（forward）

```
if (w->fused) {
    // ①②③ を 1 回で: 864 行の行列 → 864 個の答えを q, k, v に切り分ける
    matmul(s->qkv, s->xb, lw->wq, dim, dim + 2 * kv_dim);
    memcpy(s->q, s->qkv, dim * sizeof(float));
    memcpy(s->k, s->qkv + dim, kv_dim * sizeof(float));
    memcpy(s->v, s->qkv + dim + kv_dim, kv_dim * sizeof(float));
}
...
if (w->fused) {
    // ⑥⑦ を 1 回で: 前半 768 個が h1、後半 768 個が h3
    matmul(s->hb13, s->xb, lw->w1, dim, 2 * hidden_dim);
    hb = s->hb13;
    hb2 = s->hb13 + hidden_dim;
}
```

- k と v は、KV キャッシュの今の位置に `memcpy` で書き込む（288 個ずつで、時間はわずか）。
- ⑧ SwiGLU 以降は、元のコードと同じ。
- あわせて、`forward()` が層ごとの重みを「層ごとの住所の表」（`LayerWeights`）から読むように変えた。全モード共通の変更で、`-w 0〜3` の計算結果は変わらない。

計算結果が変わらないことの確認

- `matmul` は行ごとに同じ順番で足し算するので、各行の答えは元と 1 ビットも変わらない。
- `-w 0〜5` の全モード・全構成（計 560 回の実行）で、生成された文章が完全に一致することを確認した。

---

5. 測定方法

- ネイティブ Linux、電源プロファイル「パフォーマンス」に固定、AC 電源
- `stories15M.bin`、`-n 256 -t 0 -s 42 -i "Once upon a time"`
- 構成 7 通り（班の Day4 案 2 の P1, P2, P4_HT, E4, X4_2_2 ＋ E1, E2）× モード 4 通り（`-w 0, 3, 4, 5`）を、毎周ランダムな順番で 10 周

- 同じ測定を 2 回（16:05・16:09）行い、値がほぼ一致することを確認
- スクリプト: `MODES="0 3 4 5" bash core_thread_check.sh`

---

6. 結果

速さ（tok/s、16:05 の測定。16:09 もほぼ同じ）

構成	CPU	-w 0（元）	-w 3（ヒュージページ）	-w 4（＋並べ替え）	-w 5（＋まとめる）
P1	0	446	456	457	461
P2	0, 2	573	592	594	617
P4_HT	0〜3	631	667	666	**702**
E4	4〜7	491	499	500	519
X4_2_2	4, 5, 8, 9	558	571	572	599
E2	4, 5	377	386	386	397

（E1 は 10 周の中で最大 3 割ぶれたため、判断に使っていない）

まとめる効果だけを取り出す（-w 5 と -w 4 の比較、同じ周どうし、2 回の値）

構成	スレッド数	層の合計	出力層	forward
P1	1	1.016 / 1.024 倍	約 1.00 倍	1.008 / 1.008 倍
P2	2	**1.084 / 1.083 倍**	約 1.00 倍	1.039 / 1.040 倍
E2	2	**1.063 / 1.066 倍**	約 1.00 倍	1.025 / 1.029 倍
P4_HT	4	**1.117 / 1.116 倍**	約 1.00 倍	1.056 / 1.056 倍
E4	4	**1.087 / 1.094 倍**	約 1.00 倍	1.039 / 1.041 倍
X4_2_2	4	**1.100 / 1.101 倍**	約 1.00 倍	1.048 / 1.051 倍

- 層の合計は、P1 以外の構成で 10 周中 10 回速かった。
- 並べ替えだけ（-w 4 と -w 3 の比較）は、どの構成でも 1.00 倍前後で効果がなかった。速くなったのは「並べ替え」ではなく「まとめて計算したこと」による。
- 出力層は計算を変えていないので、変化なし。

P4_HT の内訳

	-w 0	-w 5
層の合計	0.801 ms	**0.670 ms（16% 減）**
出力層	0.753 ms	0.724 ms
forward	1.554 ms	**1.395 ms**

---

7. 考察

スレッドが多いほど効く理由

スレッド数	層の合計がどれだけ速くなったか（-w 5 / -w 4）
1	約 2%
2	約 6〜8%
4	約 9〜12%

1 スレッドでは待ち合わせ自体がないので、ほとんど速くならない。スレッドが多いほど、待ち合わせ 1 回の手間が大きくなり、回数を減らした効果も大きくなる。「待ち合わせの回数が減ったから速くなった」という説明と合っている。

なぜ llama2.c は最初からまとめていないのか

- llama2.c は、LLM の推論を 1 つの C ファイルで分かりやすく見せるための教育向けのプログラム。元の PyTorch のモデル（`model.py`）で Wq・Wk・Wv、W1・W3 が別々の部品として定義されている形を、そのまま C に書き写している。
- 標準のビルド（`make run`）は 1 スレッドで、待ち合わせがない。1 スレッドでは、まとめても 2% しか速くならない。
- `stories15M` は行列がとても小さい（288 × 288）ので、待ち合わせの手間の割合が大きかった。Llama 2 7B（4096 × 4096）のような大きなモデルでは、1 回の仕事が大きいので、効果は小さくなると考えられる（推測）。
- 速さを追求する推論エンジン（vLLM など）では、Q・K・V や W1・W3（gate と up）をまとめて計算する工夫が一般的に使われている。

重みを減らしたわけではない

-w 5 は、同じ 23.9MB の重みを並べ方を変えて全部読んでいる。数値を捨てたり変えたりはしていない（量子化とは別の改善）。冗長だったのは「データ」ではなく、「計算の手順の区切り方」。

---

8. 次の狙い

-w 5 で層が速くなった結果、P4_HT では **出力層（0.724 ms）の方が層の合計（0.670 ms）より時間がかかる** ようになった。次は出力層の高速化が狙い目で、例えば案２－１の「int8 で候補を絞り、候補だけ fp32 で計算し直す（2 段階の厳密な argmax）」との組み合わせが考えられる。